Reviewer Pack — Noncommutative Algebraic Growth and Tseitin Resolution Width

Entry e87da145bc32 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 00:30:18 UTC

Noncommutative Algebraic Growth and Tseitin Resolution Width

Entry ID: `e87da145bc32` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 00:30:18 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra **Field B** (complexity object): Resolution Width of Tseitin Formulas

Statement:

> For a random 3-CNF formula Φ with n variables, let A_Φ be the noncommutative algebra generated by variables x_i and relations x_i x_j = x_j x_i if clauses (i,j) are satisfied. The growth rate of dim(A_Φ_k) is Ω(2^{k/2}) for all k ≤ log n iff Φ's Tseitin formula has resolution width ≥ log n.

Rationale (proposer's reasoning):

> Noncommutative algebras encode clause interactions via generators, while resolution width measures proof complexity. The conjecture links algebraic growth rates to proof complexity via commutativity constraints, revealing structural trade-offs between algebraic dependencies and proof efficiency.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5c23080df795dc84`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=27.7s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.choice([i, -i]) for i in range(1, n + 1)]
            random.shuffle(literals)
            clause = tuple(sorted(literals))
            if clause not in clauses:
                clauses.append(clause)
        return clauses

    def is_satisfied(clause, assignment):
        return any(assignment[abs(lit)] == (lit > 0) for lit in clause)

    def generate_algebra(clauses, n):
        algebra = set()
        variables = [f'x{i}' for i in range(1, n + 1)]
        relations = [(i, j) for i in range(n) for j in range(i + 1, n)]

        def add_relation(x, y):
            if (x, y) in relations:
                algebra.add((x, y))
                algebra.add((y, x))

        for clause in clauses:
            for lit1, lit2 in itertools.combinations(clause, 2):
                var1 = variables[abs(lit1) - 1]
                var2 = variables[abs(lit2) - 1]
                add_relation(var1, var2)

        return algebra

    def dimension(algebra, k):
```

```
        if not algebra:
            return 0
        basis = set()
        for rel in algebra:
            if all(rel[i] not in basis for i in range(2)):
                basis.add(rel[0])
                basis.add(rel[1])
        return len(basis)

    n = 40
    clauses = generate_3cnf(n)

    algebra = generate_algebra(clauses, n)
    growth_rate = [dimension(algebra, k) for k in range(1, 11)]

    tseitin_width = math.log2(n)

    metric_name = "growth_rate"
    metric_value = sum(growth_rate) / len(growth_rate)
    instances_tested = len(growth_rate)
    conjecture_holds = all(x >= 2**(k/2) for k, x in enumerate(growth_rate))
    counterexample = "" if conjecture_holds else "growth_rate_mismatch"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"growth_rate_mismatch\" first_failing_seed={firs
    else:
        print("RESULT: INCONCLUSIVE no_conjecture_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
 0.0, 'instances_tested': 10, 'conjecture_holds': False, 'counterexample': 'growth_rate_mismatch'}
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
TRIAL: {'metric_name': 'growth_rate', 'metric_value': 0.0, 'instances_tested': 10, 'conjecture_holds
RESULT: FALSIFIED counterexample="growth_rate_mismatch" first_failing_seed=0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The conjecture was falsified by a counterexample showing growth_rate_mismatch in all tested instances. | next: Analyze the specific counterexample to determine how the growth rate deviation correlates with Tseitin resolution width properties

11. Audit log (LLM calls)

Total LLM calls: 15

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	48520
2	propose	ollama_remote	qwen3:8b	0	0	138171
3	propose	ollama_remote	qwen3:8b	0	0	59186
4	propose	ollama_remote	qwen3:8b	0	0	138006
5	propose	ollama_remote	qwen3:8b	0	0	164002
6	propose	ollama_remote	qwen3:8b	0	0	118121
7	preregistration	ollama_remote	qwen3:8b	0	0	31654
8	novelty	ollama_remote	qwen3:8b	0	0	27470
9	novelty	ollama_remote	qwen3:8b	0	0	14773
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15263
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9533
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10859
13	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10219
14	critic	ollama_remote	qwen3:8b	0	0	38608
15	judge	ollama_remote	qwen3:8b	0	0	23592

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 847976 ms total latency. Provider mix: {'ollama_remote': 15}

(full prompt+response transcripts available in `research/audit/e87da145bc32.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e87da145bc32.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e87da145bc32.tar.gz` (if generated)